Dongwoo (Michael) Kwon | Frontend Developer

Email : hyeonggyugwon3@gmail.com

Phone : 010-8786-9260

GitHub : @kwondongwoo0424

Website : www.kwondongwoo.com

Blog : dlog.kwondongwoo.com

A developer who grows through experimentation and failure.

Hello, I am **Dongwoo (Michael) Kwon**, **a junior developer** who believes that failure is essential for growth. I actively work with **frontend technologies** and enjoy developing with Agile methodologies and the MVP approach.
I have experience across the full development lifecycle: **service planning, design, development, deployment, and operation.**

I **continuously challenge** myself to become a better developer, learning and growing from failures. When working on projects, I maintain **product ownership** while consistently learning and improving UX. Like Nietzsche's quote, **"What does not kill me makes me stronger"**,
I use failure as a **stepping stone** to become a better developer.

Skills

Languages	**JavaScript**, **TypeScript**, Java, Python
Frontend	**React**, Next.js, Vue.js, **styled-components**, tailwind CSS, Bootstrap, Tanstack Query, Zustand, Zod, Axios, Storybook, Electron
Backend	Node.js, Express.js, Spring Boot
Deployment & Tools	Git, Github, Figma, Postman, Docker, Supabase, Vercel, Netlify, Github Pages

Intern

XL8 Inc. 2026.01.06. ~ 2026.02.27. (8 weeks)

Contributed to migrating a company landing page from Webflow to a Supabase · Plasmic stack.

- Migrated Webflow CMS data to Supabase database structure, improving data management workflows
- Implemented data integration pipeline fetching Supabase DB content in Next.js and exposing it as CMS data within Plasmic
- Applied localization architecture for multi-language support and implemented responsive design

Projects

Featured Project

Sync Desktop

Team Project
7 members

AI-powered project management SaaS for collaboration between non-developers and developers

React | TypeScript | Electron | styled-components | TanStack Query | Zustand

Desktop app development / UI design / Design system implementation / Landing page development / SSE-based real-time term interpretation feature development

Dlog

Personal Project
Solo

Personal Tech Blog

Vue.js | TypeScript | Bootstrap | Sanity CMS | TanStack Query

FSD Architecture / Responsive Design / Dark Mode Implementation

Side Projects

DIA

Team Project
4 members

GPA calculation service for Daegu Software Meister High School

React | TypeScript | styled-components | React Context API

Grade calculation page development / Grade calculation algorithm modularization

ColorVerse

Personal Project
Solo

Web service for color palette recommendations

React | styled-components | Express.js | OpenAI API

Express.js server implementation / OpenAI API integration and prompt engineering

Flick

Team Project
5 members

Digital currency-based real-time transaction and settlement platform for school festival booth operations

Next.js | Tailwind CSS | TypeScript | pnpm

Monorepo Architecture / Admin page real-time booth ranking system development

Personal Web Portfolio

Personal Project
Solo

Next.js | Tailwind CSS | TypeScript | next-intl

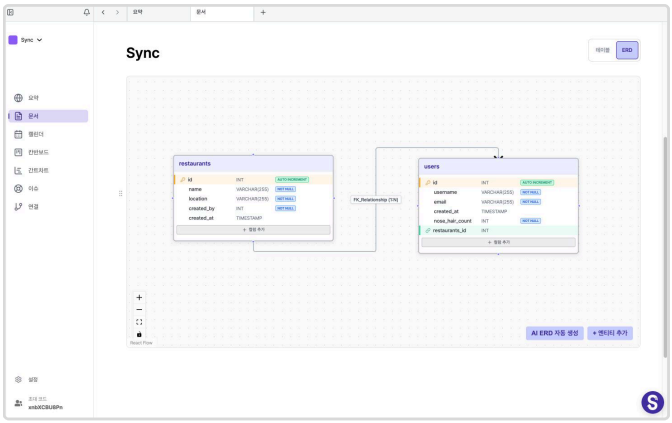
Korean/English Multilingual Support / Search Engine Optimization (SEO)

Featured Project

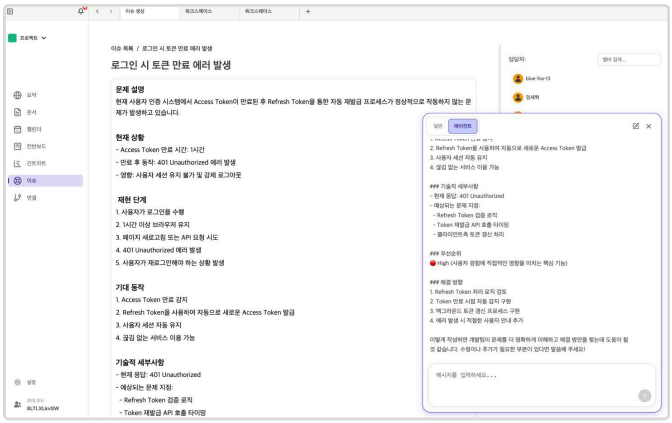
Sync Desktop

Mar 2025 - Oct 2025

AI-powered project management SaaS for collaboration between non-developers and developers



Document Page - Auto ERD generation based on functional specifications



DeepSync - Automatic issue content correction

Overview

An AI-powered project management desktop app designed to bridge the communication gap and collaboration inefficiencies caused by technical knowledge disparities between developers and non-developers such as designers and planners. The core goal is to leverage AI to fill knowledge gaps and provide a real-time collaborative environment for cross-functional teams.

Key Challenges

- Non-developers struggle to articulate requirements due to limited technical knowledge
- Developers miss business context, leading to communication overhead
- Frequent misunderstandings, time waste, and repetitive work in collaboration

Solution

Enhance non-developers' technical understanding through AI while minimizing communication costs with real-time collaboration features.

Key Features

- Drag-based technical term interpretation for non-developers
- AI chatbot with project context awareness for personalized assistance
- Real-time collaborative
- Markdown documentation and AI-powered ERD generation from functional specifications

Team

7 members total

Frontend: 4 (including myself) | Backend: 3 | **Design**: 2 (including myself)

My Role: Frontend Developer & UI/UX Designer

Tech Stack

React

Chosen for rapid development of dynamic, stateful user interfaces

TypeScript

Ensures type safety essential for team collaboration and maintainability

Electron

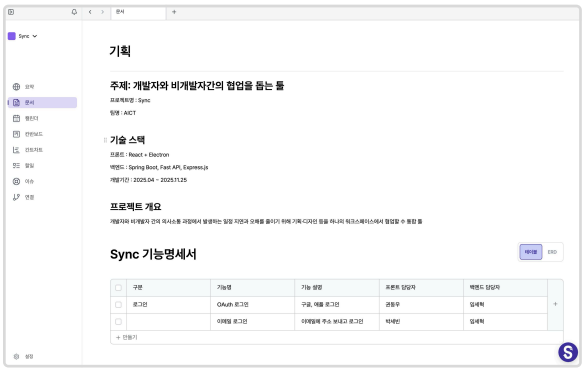
- Leverages familiar web technologies for desktop app development
- Provides cross-platform support for consistent UX across macOS and Windows
- Maintains browser-like development workflow for enhanced productivity

styled-components

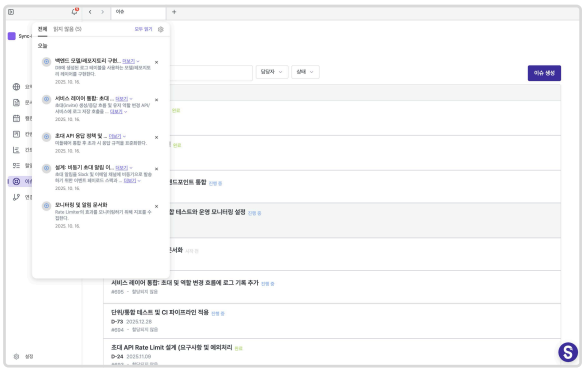
Manages component-level styling with consistency

Key Activities & Responsibilities

UI/UX Design



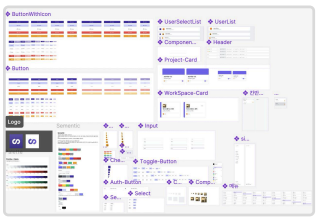
Document Page



Issue Page

Designed user-centric interfaces leveraging **Figma's Auto Layout** feature

Design System Development (sync-design-system)



Design System Implementation (Figma-based)

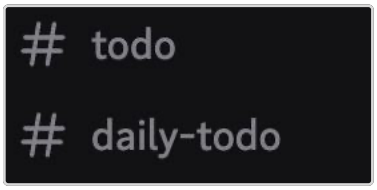
sds published on npmjs

Inconsistent UI components hindered maintainability and reusability.

- Extracted 15 reusable common components
- **Published** as **npm package** (@sync-design-system) for team-wide sharing
- Provided **component documentation** and **usage guidelines** via **Storybook**

→ **Improved development efficiency** and **UI consistency** through systematic component design

Agile Methodology Implementation



Daily todo tracking (via Discord)

Weekly sprint retrospectives

During a 7-month team project, the initial 2 months were spent without clear milestones, resulting in major feature delays and difficulty tracking progress across team members.

- **Implemented weekly sprints**: Thursday demos and retrospectives, Friday sprint planning
- Managed backlog, sprint boards, and retrospectives via Notion
- Shared daily reminders and blocking issues through Discord

→ Gained experience improving schedule predictability and team productivity in long-term projects

Key Activities & Responsibilities

UX Improvements

Reduced Issue Page Loading Time with React Query Caching

API re-fetching occurred on every navigation between issue list and detail pages
→ Average 2-second loading time with repeated requests increasing server load

- Implemented **React Query** for automatic issue **data caching**
- Set 2-minute staleTime: fresh data displays instantly without re-fetching (short staleTime used due to frequently changing issue data)

→ **Improved performance** and **user experience** simultaneously through caching strategy

Improved Task Check Responsiveness with Optimistic Updates

0.5-1s delay occurred while waiting for server response
when checking tasks as complete → degraded user experience

Implemented React Query **optimistic updates** for instant UI response

- **Reduced perceived response time** from **800ms to 0ms** on average
- Automatic rollback on network failure with error toast notifications

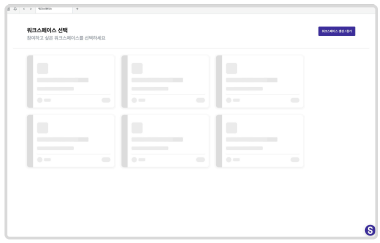
Task list with
optimistic updates applied

Removed Unnecessary Skeleton UI

Skeleton UI on workspace cards caused brief flash as it appeared and disappeared during loading

- Chrome DevTools showed average API response time of 180ms
- Skeleton UI vanished immediately after appearing,
  creating visual noise

→ Removed skeleton UI and rendered content immediately upon data arrival for a **clean, flicker-free loading experience**

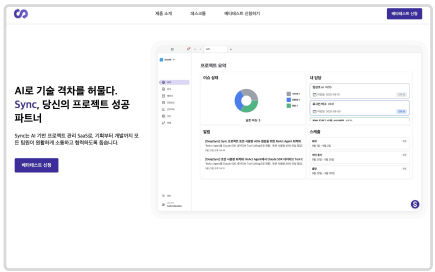


Workspace page
with skeleton UI applied

Beta Test Landing Page (sync introduce) Development

- Built beta signup page using GitHub Pages with custom domain
- Provided Windows/macOS installers via GitHub
  Releases-based deployment

→ Gained experience establishing and executing an **efficient beta distribution strategy** for product validation



Sync-introduce

Troubleshooting

Optimizing Excessive Re-rendering in SSE Streaming Responses

Problem

While implementing real-time AI word explanations via SSE when users drag text, the UI stuttered and lagged during response display.

Root Cause

Excessive re-rendering from setState calls on every chunk received

- n chunks received → n re-renders triggered
- Virtual DOM generation and diffing process executed per render

→ Performance degradation

```
onChunk: (chunk) => {
  // 현재 처리 중인 단어가 맞는지 확인
  if (currentWordRef.current !== word) {
    return; // 다른 단어의 응답이면 무시
  }

  // 매 청크마다 즉시 상태 업데이트
  setDefinition(prev => prev + chunk);
},
```

Original code

Solution Process

Considered two approaches: Throttling vs. Debouncing

1. Throttling
   - Pros: Consistent, smooth typing animation with regular render intervals
   - Cons: Unnecessary renders occur even when no new chunks arrive
2. Debouncing
   - Delays rendering while chunks arrive continuously
   - Executes single render only after no new chunks for set duration

While Throttling provided smoother animation, it had lingering performance issues from unnecessary renders. Debouncing, though slightly less smooth in typing effect, drastically reduced render count and most effectively resolved UI stuttering.

Final approach: Debouncing selected

Solution

Debouncing + useRef Buffering Pattern

1. useRef as intermediate buffer → accumulates data without re-rendering
2. 50ms Debouncing → delays rendering during continuous chunk arrival
3. Cancel previous timer → setState called only once based on last chunk

→ Chunks accumulate in buffer in real-time, but screen updates only when no additional chunks arrive for 50ms

```
onChunk: (chunk) => {
  // 현재 처리 중인 단어가 맞는지 확인
  if (currentWordRef.current !== word) {
    return; // 다른 단어의 응답이면 무시
  }

  definitionRef.current += chunk;

  if (updateTimerRef.current) {
    clearTimeout(updateTimerRef.current);
  }

  updateTimerRef.current = setTimeout(() => {
    // 다시 한번 현재 단어 확인
    if (currentWordRef.current === word) {
      setDefinition(definitionRef.current);
    }
  }, 50);
},
```

Code with Debouncing applied

Results & Learnings

- Resolved UI stuttering by reducing setState call frequency
- Gained deep understanding of React rendering mechanisms
- Learned to clearly differentiate Throttling vs. Debouncing and apply appropriately based on context

Reflection & References

Reflection

Through this 7-month team project, I learned that teamwork doesn't happen automatically. We faced conflicts from different experiences and struggled when intentions weren't fully understood. However, open communication quickly resolved misunderstandings, and I learned that the willingness to understand each other matters more than problem-solving itself. This project taught me the value of growing together as a team, not just building features.

Achievements

FIX 2025 Korea ICT Convergence Expo Booth Operation	Oct 22-25, 2025
3rd Place, 2025 ITCE Project Competition	Oct 24, 2025

References

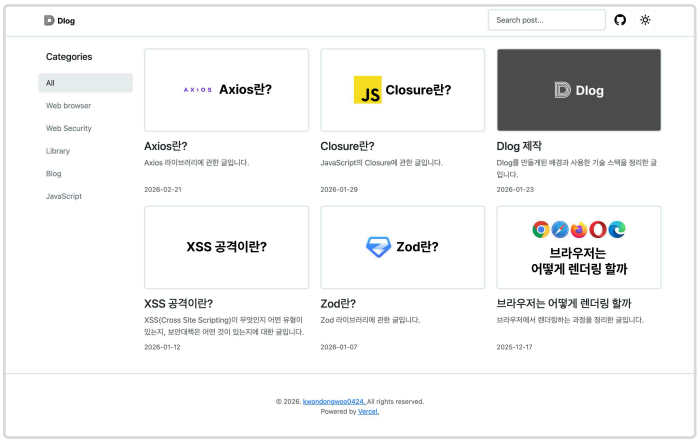
Sync-Desktop Source Code:	https://github.com/AICT-SYNC/sync-desktop
Sync Design System Source Code:	https://github.com/AICT-SYNC/sync-design-system
Sync Design System NPM:	https://npmjs.com/package/sync-design-system
Sync Introduce Source Code:	https://github.com/AICT-SYNC/sync-introduce
sync-saas Website (Service Ended):	https://sync-saas.com

Featured Project

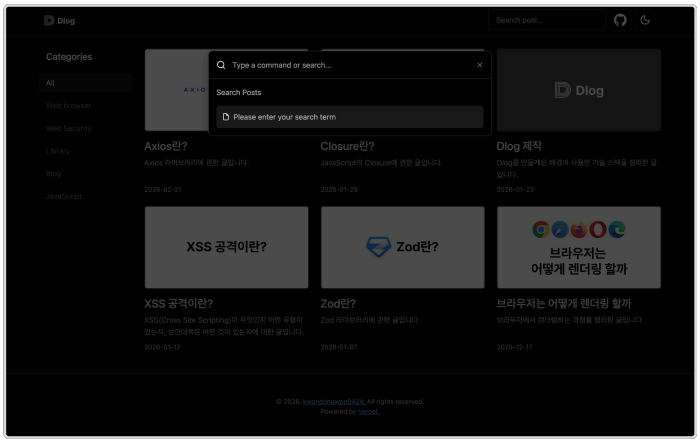
Dlog

Personal Tech Blog

Jan 27, 2026 – Present



Home Screen — Light Mode



Home Search Screen — Dark Mode

Overview

Previously managed TIL (Today I Learned) entries by committing directly to GitHub.
To overcome the limitations of cumbersome file management and the lack of category and search functionality — and to gain hands-on experience with Vue.js and Bootstrap — I developed a personal blog powered by Sanity CMS. It enables content management without a dedicated backend and is currently in active use for documenting my learning progress.

Team

Solo Project

Tech Stack

Vue.js | TypeScript | Bootstrap | Sanity CMS | TanStack Query

Reflection

Beyond simply implementing features, this project allowed me to experience the full lifecycle of frontend development — from architectural design to SEO and production operations. Applying FSD architecture in particular cultivated a habit of thinking critically about code structure, and seeing real user traffic data firsthand reinforced that development doesn't end at launch — it extends into ongoing operations.

References

Dlog Web Page : dlog.kwondongwoo.com
Dlog-web Source Code : github.com/kwondongwoo0424/Dlog
Dlog-sanity Source Code : github.com/kwondongwoo0424/Dlog-sanity

Key Activities & Responsibilities

Feature-Sliced Design (FSD) Architecture Implementation

Faced unclear component dependencies and increasing difficulty locating code as features grew.

- Separated responsibilities into pages / widgets / entities / shared layers
- Enforced unidirectional dependency rules where only upper layers reference lower layers
- Improved feature-level cohesion, making it intuitive to locate where to add or modify code

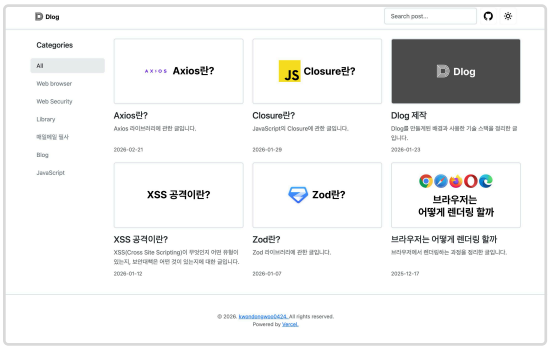
→ Achieved **maintainability** and **scalability** through a consistent architectural structure

Responsive layout delivering device-optimized UX across all screen sizes

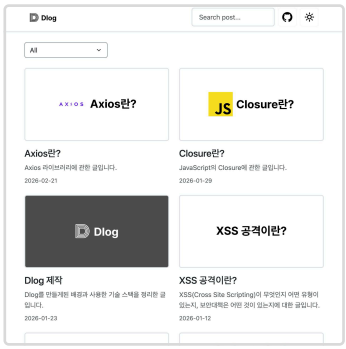
Applying the same UI across desktop and mobile resulted in degraded usability due to screen size differences.

- Desktop: Persistent sidebar displaying category list for quick navigation
- Mobile: Replaced sidebar with a category dropdown to maximize screen space efficiency
- Implemented component branching based on breakpoints using CSS media queries

→ Delivered device-appropriate UX tailored to each platform's characteristics



Desktop

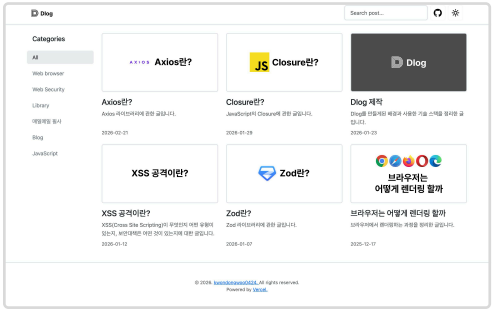


Mobile

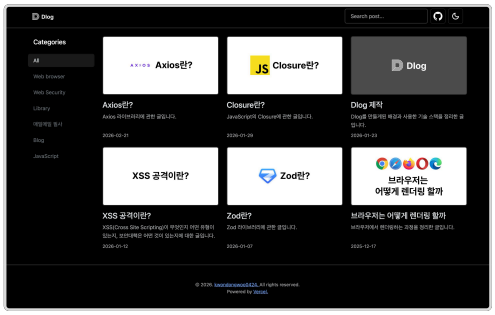
Dark Mode Implementation — System Preference Detection & User Setting Persistence

- Automatically detected system dark mode preference via prefers-color-scheme media query
- Persisted user selection in localStorage to retain theme preference across revisits
- Applied global theme switching via Bootstrap data-bs-theme attribute

→ On first visit, theme defaults to system preference; subsequent visits prioritize the user's saved setting



Light Mode



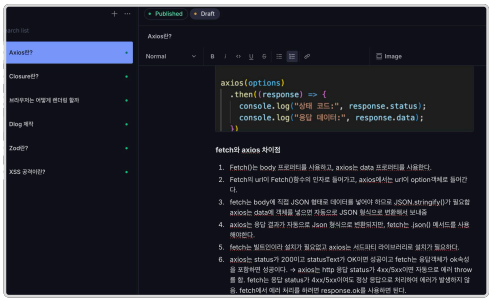
Dark Mode

Key Activities & Responsibilities

Headless CMS (Sanity) Content Management Environment — Backend-Free Architecture

Needed a structure to manage content while minimizing development time and operational costs.

- Introduced Sanity Headless CMS to establish a content management environment without a dedicated server
- Content created, edited, or deleted in Sanity Studio reflects instantly without redeployment
- Implemented flexible data queries including category filtering and latest-first sorting via GROQ queries



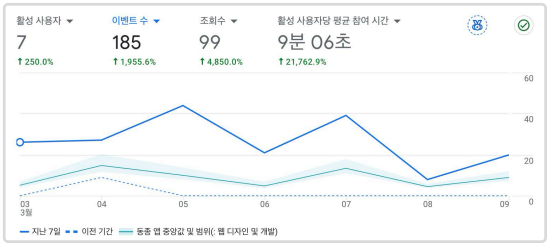
Sanity Studio — Post Editor Screen

→ Eliminated server development, deployment, and maintenance overhead — enabling full focus on frontend development while reducing operational costs

Production Deployment & User Traffic Monitoring

Took the project beyond development by operating it as a live service and tracking real user traffic.

- Published 7 blog posts currently live on the platform
- Analyzed user acquisition data via Google Analytics dashboard



Google Analytics Dashboard

→ **Gained hands-on production experience** through active content creation and service operation

Troubleshooting

Search Engine Post Page Non-Indexing Issue

Problem

After deploying the website, monitoring via Google Search Console and site:dlog.kwondongwoo.com revealed that only the homepage (/) was being indexed, while all post detail pages (/post/{slug}) were completely absent from search engine index.

Root Cause

The statically authored public/sitemap.xml contained only the homepage URL (/).
Since Googlebot relies on the sitemap to discover crawl targets, post pages were never registered — preventing the search engine from recognizing those pages entirely.

Solution Process

Evaluated two approaches for including post URLs in the sitemap.

Option 1 — Build-Time Sitemap Generation

- Automatically generates a sitemap by querying the Sanity post list at deployment
- New posts published after the build are excluded from the sitemap until the next deployment
- Requires triggering a redeployment on every content update, resulting in unnecessary repeated deployments

Option 2 — Dynamic Sitemap via Serverless Function

- Generates XML in real time by querying Sanity for posts and categories on each request
- New posts are immediately reflected in the sitemap without redeployment

Since the Sanity CMS architecture decouples content authoring from deployment, choosing Option 1 would require a redeployment every time a new post is published — effectively nullifying the core benefit of adopting a CMS. Option 2 was selected on this basis.

Solution

Authored a Vercel serverless function (api/sitemap.xml.ts) to handle /sitemap.xml requests.

The function queries Sanity for all posts (slug, publishedAt, _updatedAt) and categories (slug), then dynamically generates and returns a Sitemap XML covering the homepage, category pages, and post detail pages.

An additional issue was discovered where the existing public/sitemap.xml static file took precedence over the serverless function in Vercel, causing it to be overridden. To resolve this, the static file was deleted and a rewrites rule was added to vercel.json to explicitly route all /sitemap.xml requests to the serverless function.

Troubleshooting

Results & Learnings

- After registering the sitemap in Google Search Console, post detail pages began indexing sequentially. New posts are now automatically reflected in the sitemap on the next crawl without requiring redeployment.
- Confirmed that selecting a sitemap strategy aligned with the content update workflow is more critical than defaulting to a static file approach.
- By comparing the trade-offs between static and dynamic sitemap generation,
  gained a deeper understanding of the importance of designing with both SEO considerations and infrastructure architecture in mind.

Side Project

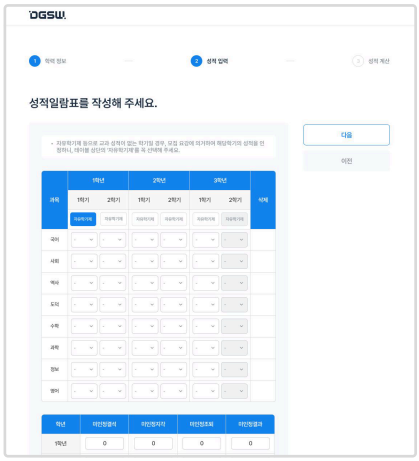
DIA

Aug 9 - Sep 9, 2025

GPA Calculation Service for Daegu Software Meister High School

Key Responsibilities

- GPA calculation page development
- Grade calculation algorithm modularization



DIA Grade Entry Interface

Tech Stack

React | TypeScript | styled-components

References

Frontend Source Code: https://github.com/EntryCNS/DIA
Website URL: https://trydgsw.com

Overview

A simple and intuitive web service that calculates total admission scores based on the official grading criteria of the first-round selection process at Daegu Software Meister High School, allowing students to input their academic grades and instantly receive their projected scores.

Team

4 members total
Frontend: 4 (including myself)

Key Learnings

Global State Management with React Context API

Developed 3-step form flow: student type selection → grade input → result confirmation

- Props passed through 5-6 components (5-level depth)
- Required modifying all 6 files when adding new data fields

→ **Centralized global state management** with **React Context API**

- Real-time sync: shared state across multiple components
- **Eliminated Props Drilling**: 5-level passing → direct access

→ Built systematic state management architecture for improved maintainability

```
Before (Props Drilling)

[App] - grades, setGrades, freeSem, attendance... (12개 props)
  ↓
[Router] - (12개 props 그대로 전달)
  ↓
[StudentWritePage] - (12개 props 그대로 전달)
  ↓
[Body] props만 전달, 사용 안함 - (12개 props 그대로 전달)
  ↓
[WriteGrade] - grades, setGrades 사용
  ↓
[WriteGradeListItem] - grades 사용
```

→

```
After (Context API)

[App]
  └ [ScoreProvider] 전역 상태
        ├ [StudentWritePage] → useScore()
        ├ [WriteGrade] → useScore()
        └ [WriteGradeListItem] → useScore()
```

Why React Context API over Redux/Zustand

- Project requirements were simple ("share form data across pages"), not requiring Redux's action/reducer structure or Zustand's selector-based architecture
- Low state change frequency meant Redux/Zustand's granular state separation wasn't needed; Context API provided sufficient performance

Side Project

ColorVerse

Color Palette Recommendation Service

Jun 21-25, 2025

Key Responsibilities

- Express.js server implementation

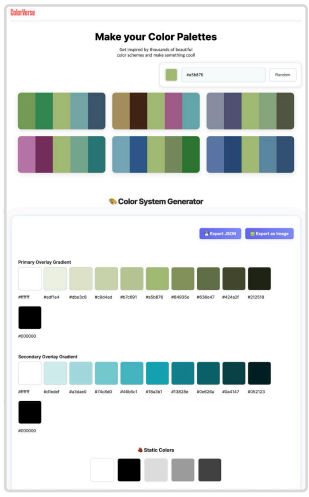
Tech Stack

React | styled-components | Express.js | OpenAI API

References

Source Code: https://github.com/kwondongwoo0424/ColorVerse

Notion Docs: https://buly.kr/GvoBKj0



Main interface

Overview

A web service that helps designers and developers quickly establish consistent color systems and discover harmonious color combinations with AI assistance.

Team

Solo project

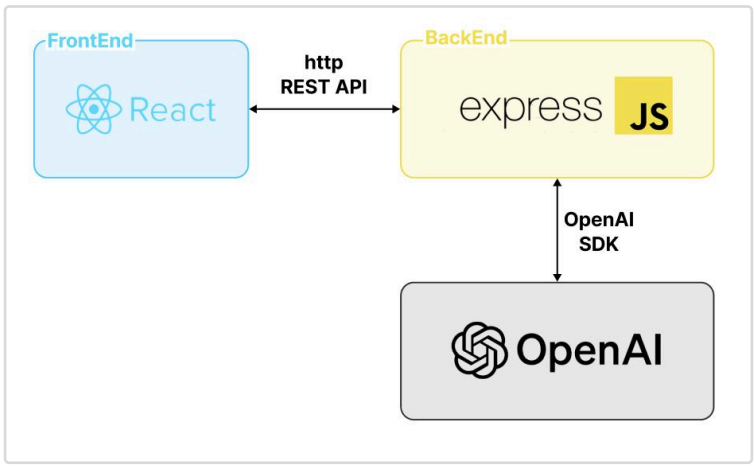
Key Learnings

RESTful API Server with Express.js

- Integrated OpenAI API via /api/chat POST endpoint
- Implemented frontend-backend communication with CORS configuration

OpenAI API Integration & Prompt Engineering

- Integrated GPT-3.5-turbo model using official OpenAI Node.js SDK (openai)
- Implemented complete API flow: authentication → request composition → response handling



Side Project

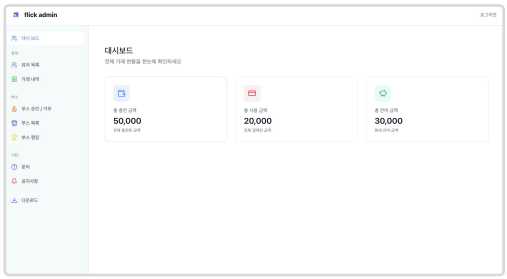
Flick

May 5-20, 2025

Digital Currency-Based Real-Time Transaction and Settlement Platform for School Festival Booth Operations

Key Responsibilities

- Admin page development



Flick Admin Dashboard Page

Tech Stack

Next.js | tailwind CSS | TypeScript

References

Web Source Code: https://github.com/EntryCNS/flick-web

Overview

Flick is a point-based transaction service designed for school festival booth payments.
The previous v1, developed by seniors, had several pain points in its QR code payment flow.
Flick v2 was rebuilt from the ground up to address these issues and deliver a significantly improved user experience.

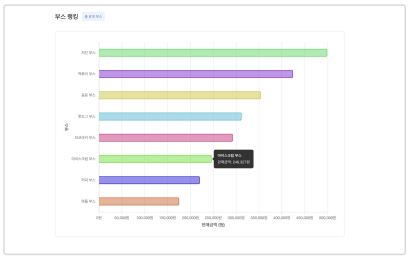
Team

5 members total
Frontend: 2 (including myself) | Backend: 2 | Full-Stack: 1
My Role: Frontend Developer

Web Development

Developed a real-time booth ranking system

- Built a real-time revenue ranking visualization using Chart.js and WebSocket
- Implemented WebSocket reconnection logic to ensure stable and reliable real-time data communication



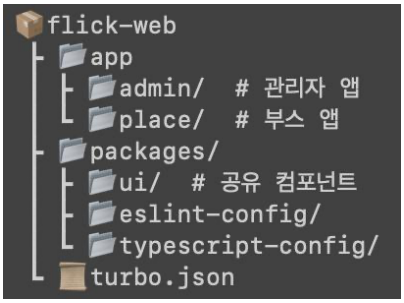
Admin Ranking Page

Key Learnings

Codebase Integration and Management Efficiency

Adopted a Turborepo- and pnpm Workspace–based **monorepo**

- @repo/ui: Centralized shared components to eliminate code duplication
- pnpm hard links: Reduced node_modules storage usage
- workspace:* protocol: Automated dependency management across packages



Flick-web Project Structure

→ Gained hands-on experience achieving both productivity and consistency in a multi-project environment

Side Project

Personal Web Portfolio

Dec 28, 2025 – Present

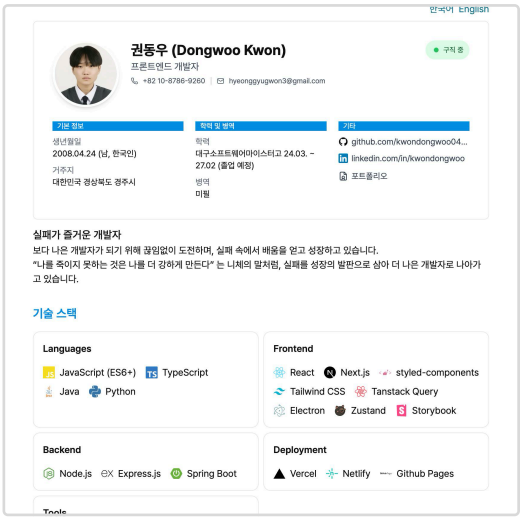
Tech Stack

Next.js | tailwind CSS | TypeScript | next-intl

References

Frontend Source Code
: github.com/kwondongwoo0424/portfolio

Website URL:
: kwondongwoo.com



Home Screen

Overview

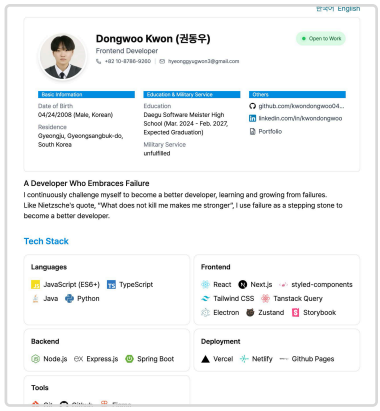
Feeling the limitations of my existing Notion-based portfolio — including restricted design flexibility and lack of SEO support — I built a personal portfolio website from scratch using Next.js App Router.

Web Development

i18n Korean/English Multilingual Support

Adopted next-intl for compatibility with App Router and Server Components.

- Implemented locale-based routing via dynamic [locale] segments, handling /ko and /en URL paths per language
- Configured automatic language redirect based on Accept-Language header using Vercel + next-intl
- Built language toggle using usePathname() to detect the current path and swap only the locale segment, enabling seamless switching without full page reload



Home Screen /en Route

Search Engine Optimization (SEO)

- Adopted Next.js with SSR to ensure search engine crawlers can index meta tags and content without executing JavaScript
- Utilized generateMetadata() to generate locale-specific meta tags separately for /ko and /en routes
- Applied hreflang alternates including x-default to help search engines accurately identify language-specific pages
- Dynamically generated sitemap.xml and robots.txt via sitemap.ts and robots.ts API routes

Lighthouse Audit Results

Certifications & Qualifications

Craftsman Information Processing	Dec 24, 2025
23rd TOPCIT (Test of Practical Competency in ICT) - 527 points (Level 3)	May 24, 2025
TOEIC Bridge - 76	Dec 23, 2024

Awards

Academic Excellence Award in Korean History	Dec 22, 2025
3rd Place, 2025 ITCE Project Competition	Oct 24, 2025
Academic Excellence Award in Physical Education	Jul 22, 2025

Activities

Silicon Valley Online Internship — XL8 Inc.	Jan 6 – Feb 27, 2026
SW Meister High School Joint Hackathon Participant	Nov 5-7, 2025
FIX 2025 Korea ICT Convergence Expo - Booth Operator	Oct 22-25, 2025
School Software Vibe Coding Hackathon Participant	Jul 15-16, 2025
Narshar Project Participant - Sync Desktop	Mar 17 - Oct 25, 2025
1:1 Native English Online Program	May 13 - Dec 23, 2025 (Ongoing)
Library Committee Head	Dec 2024 - Present
Admissions Interview Volunteer	Oct 25, 2024
Narshar Project Participant	Aug 20 - Dec 26, 2024
School Software Hackathon Participant	Jul 17-18, 2024
1:1 Native English Online Program	May 8, 2024 - Jan 8, 2025
Coding Test Competition Participant	Apr 17, 2024
Smarteen App Challenge Participant	Apr 2, 2024
CNS Programming Club Member	Mar 14, 2024 - Present

Thank you for reading to the end.
I still have much to learn, but I am committed to becoming a developer who grows one step at a time, every day.